

```
language=Python, escapechar= basicstyle=, breaklines=true, keywordstyle=blue,  
commentstyle=gray, stringstyle=red, showstringspaces=false, frame=single,  
rulecolor=black!30
```

Contents

List of Figures

List of Tables

Chapter 1

GUI Description: Complete Reference

File: `cardiac_sim/gui/main_window.py`

1.1 Main Window Architecture

The application window has three main regions:

[Diagram - text description in document]

Figure 1.1: Main window layout. Toolbar at top; left sidebar with parameter/pathology panels; central area for ECG display.

1.1.1 Toolbar

Located at the top, the toolbar provides:

1. **Run/Pause button:** Toggle simulation on/off
 - Enabled: simulation running, ECG updating continuously
 - Disabled: simulation paused, ECG frozen
 - Default: running
2. **Reset button:** Clear ECG history and restart simulation from $t = 0$
 - Clears all buffers
 - Reinitializes engine
 - Restarts timer
3. **Engine selector combo box:** Choose between and

- Option 1: “Phase 1/2 – Conduction Graph” (default)
 - Option 2: “Phase 3 – FHN Cable”
 - Selecting triggers engine swap with no parameter loss
 - Implementation: `_on_engine_changed` in `main_window.py`
4. **Heart rate display:** Reads parameters from engine and shows current HR
- Format: “HR: XX bpm”
 - Updated every GUI frame (typically 30–60 Hz)

1.2 Left Sidebar: Parameter Panel

File: `cardiac_sim/gui/parameter_panel.py`

1.2.1 Overview

The left sidebar contains a hierarchical parameter tree. Each parameter is a **named value** with:

- **Name:** descriptive label
- **Type:** numeric (int, float), boolean, or choice
- **Default value:** physiological normal
- **Minimum/maximum:** physiologically plausible range
- **Units:** seconds, milliseconds, dimensionless, etc.
- **Current value:** user-adjustable

The tree is organized into categories (sections) for clarity:

```
blackParameters:
+- SA Node
| +- Cycle Length (ms)
| +- Ectopic Trigger Rate (beats/min)
| - [other SA parameters]
+- AV Node
| +- AV Delay (ms)
| - [others]
+- Refractory Periods
```

```

| +- Atrial (ms)
| +- Ventricular (ms)
| - [others]
+- Phase 3 Model (if Phase 3 selected)
| +- Cable Diffusion Atrial (units)
| +- Cable Diffusion Ventricular (units)
| - Time Scale Tau (ms)
- [other categories]

```

1.2.2 Complete Parameter List with Ranges and Meanings

	l	c	c	p3cm	
Parameter Name	Min	Default	Max	Meaning	
SA Node Section					
Cycle Length (ms)	300	857	2000	Time between SA fires; inverse of heart rate	
Ectopic Trigger Rate (bpm)	0	0	200	Ectopic atrial pacemaker rate if enabled	
Atrial Section					
RA Conduction Delay (ms)	0	50	200	Extra delay in right atrium	
LA Conduction Delay (ms)	0	50	200	Extra delay in left atrium	
RA-LA Conduction Delay (ms)	0	100	300	RA to LA propagation time	
AV Node Section					
AV Delay (ms)	50	100	300	Programmed AV node delay	
AV Conductance	0	1.0	1.0	0 = complete block; 1 = normal conduction	
AV Refractory Period (ms)	200	300	400	Minimum interval before re-excitation	
Ventricular Section					
RV Activation Delay (ms)	0	20	100	Extra delay in right ventricle	
LV Activation Delay (ms)	0	30	100	Extra delay in left ventricle	
SEPT Activation Delay (ms)	0	10	50	Septal activation delay	
Septal Conductance	0	1.0	1.0	0 = block septal conduction	
RBB Conductance	0	1.0	1.0	0 = complete RBBB; 1 = normal	
LBB Conductance	0	1.0	1.0	0 = complete LBBB; 1 = normal	
Ventricular Refractory Period (ms)	200	250	350	Ventricular refractoriness	
Phase 3 Model (if enabled)					
Cable Time Scale τ (ms)	5	15	30	FHN time constant; affects overall speed	
Atrial Diffusion Coefficient	0.5	2.88	10.0	Atrial cell-to-cell coupling	
Ventricular Diffusion LBB	50	72	150	His-LBB-LV diffusion (fast pathway)	
Ventricular Diffusion LV	1	3.0	20	LV myocardial diffusion (slower)	
ECG Display					
Display Window (s)	2	10	30	Horizontal axis span	
Amplitude Scale (mV/div)	0.1	0.5	2.0	Vertical scale; standard ECG = 0.5 mV/div	

1.2.3 Parameter Data Model

File: cardiac_sim/core/parameter_model.py

```

black@dataclass
    """Central holder of all tunable parameters."""

    # SA node
    sa_node: SANodeParams

```

clas

```
# Conduction delays
atrial: AtrialParams
av_node: AVNodeParams
ventricular: VentricularParams

# Refractory periods (Phase 1/2)
refractoriness: RefractoryParams

# Phase 3 parameters
cable_model: CableModelParams
```

```
# ECG display
ecg_display: ECGDisplayParams
```

Each parameter group is a dataclass with validation:

```
black@dataclass
class AVNodeParams:
    av_delay_ms: float = 100.0    # Normalized to 100 ms
    conductance: float = 1.0      # 0-1 scale: 0=block, 1=normal
    refractory_ms: float = 300.0

    def __post_init__(self):
        """Validate ranges."""
        assert 50 <= self.av_delay_ms <= 300, "AV delay out of range"
        assert 0 <= self.conductance <= 1, "Conductance must be 0-1"
```

1.3 Left Sidebar: Pathology Panel

File: cardiac_sim/gui/pathology_panel.py

1.3.1 Overview

The Pathology Panel is a **list of checkboxes**, each representing a pathological condition. When checked, the condition is applied to the current engine.

1.3.2 Available Pathologies

Pathology Name		Description and Implementation
	AV Blocks	

First-Degree AV Block	Increase AV delay by 50 ms (total 150 ms). PR interval prolonged; QRS unchanged.
Second-Degree AV Block (Mobitz I)	Progressively increase AV delay; eventually a beat is dropped. Not implemented in current version.
Second-Degree AV Block (Mobitz II)	Randomly drop 1 in N beats from AV conduction.
Third-Degree AV Block (Complete)	Set AV conductance = 0. Ventricular escape rhythm emerges (junctional or idioventricular).
Bundle Branch Blocks	
Right Bundle Branch Block (RBBB)	Set RBB conductance = 0. RV is activated late via collateral LV conduction; QRS widened, characteristic terminal R' in V1.
Left Bundle Branch Block (LBBB)	Set LBB conductance = 0. LV is activated late via collateral RV conduction; QRS widened, broad notched deflection in lateral leads.
Atrial Pathologies	
Atrial Fibrillation	Replace orderly BFS with chaotic re-entrant activation. Atrial rate 300–600 bpm; erratic baseline; absent P waves.
Atrial Flutter	Regular re-entrant circuit; atrial rate 250–350 bpm; regular “sawtooth” pattern visible in II, III, aVF.
Sinus Bradycardia	Increase SA cycle length to 1200 ms (50 bpm). Normal QRS and T morphology; just slower rate.
Sinus Tachycardia	Decrease SA cycle length to 400 ms (150 bpm). Normal morphology; faster rate.
Ventricular Pathologies	
Premature Ventricular Contraction (PVC)	Trigger ectopic ventricular activation at preset time (usually 600 ms post-SA). Produces wide QRS; T-wave opposite polarity.
Ventricular Tachycardia	Repetitive PVCs at rate 120–250 bpm. Rapid wide complexes; may degenerate to VF.
Ventricular Fibrillation	Chaotic ventricular activation; no organized QRS; baseline appears “fine” or “coarse” fibrillary waves.
Accessory Pathways	
Wolff-Parkinson-White (WPW)	Add accessory pathway bypassing AV node; early ventricular pre-excitation creates short PR and delta wave.
Atrioventricular Reentrant Tachycardia (AVNRT)	Dual AV node pathways; setup conditions for re-entrant circuit; produces regular narrow-complex tachycardia.

1.3.3 Pathology Plugin System

File: `cardiac_sim/pathology/__init__.py` and `cardiac_sim/plugins/`

All pathologies are implemented as **plugins**—modular classes inheriting from an abstract base:

```
blackclass AbstractPathologyPlugin(ABC):
    """Base class for all pathologies."""

    @property
    @abstractmethod
    def name(self) -> str:
        """Pathology name for UI."""
```

```
pass
```

```
@abstractmethod
```

```
def apply(self, engine: SimulationEngine, params: SimulationParameters) -> None:
    """Apply pathological modifications to engine and/or parameters."""
    pass
```

```
@abstractmethod
```

```
def unapply(self, engine: SimulationEngine, params: SimulationParameters) -> None:
    """Restore normal state."""
    pass
```

Example implementation (RBBB):

```
black# File: cardiac_sim/plugins/bundle_blocks.py
```

```
class RBBBPathology(AbstractPathologyPlugin):
```

```
    @property
```

```
    def name(self) -> str:
        return "Right Bundle Branch Block"
```

```
    def apply(self, engine: SimulationEngine, params: SimulationParameters) -> None:
        # Store original for later restoration
        self._original_rbb_conductance = params.ventricular.rbb_conductance
```

```
        # Apply block
        params.ventricular.rbb_conductance = 0.0
```

```
        # Some engines may need explicit reinitialization
        engine.apply_pathology(self)
```

```
    def unapply(self, engine: SimulationEngine, params: SimulationParameters) -> None:
        # Restore
        params.ventricular.rbb_conductance = self._original_rbb_conductance
        engine.apply_pathology(None) # Clear pathology
```

1.3.4 Pathology GUI Interaction

When the user checks a pathology checkbox:

1. GUI calls `apply` on the plugin

2. Plugin modifies parameters and/or engine state
3. Engine is reinitialized with new parameters (seamlessly)
4. ECG display updates to show new morphology

When unchecked:

1. GUI calls `unapply`
2. Original parameters restored
3. Engine reinitialized
4. ECG returns to normal

1.4 Central Display: ECG Plotter

File: `cardiac_sim/gui/ecg_display.py`

1.4.1 Overview

The ECG display is a custom PyQt6 widget using **pyqtgraph** for high-performance waveform rendering.

```
blackclass ECGDisplayWidget(pg.GraphicsLayoutWidget):

    def __init__(self, parent=None):
        super().__init__(parent)

        # Create 12 plot items (one per lead)
        self._plots = {}
        for lead_name in LEAD_NAMES: # ['I', 'II', ..., 'V6']
            plot = self.addPlot(title=lead_name, axisItems={'bottom': TimeAxis()})
            curve = plot.plot(pen='blue')
            self._plots[lead_name] = {'plot': plot, 'curve': curve, 'data': []}

    def update_ecg(self, ecg_sample: ECGSample) -> None:
        """Append new sample and refresh display."""
        for i, lead_name in enumerate(LEAD_NAMES):
            self._plots[lead_name]['data'].append(ecg_sample.leads[i])

        # Keep only last N samples (for time window)
```

```

        if len(self._plots[lead_name]['data']) > self.max_samples:
            self._plots[lead_name]['data'].pop(0)

        # Update plot
        self._plots[lead_name]['curve'].setData(
            self._plots[lead_name]['data'],
            pen=pg.mkPen('blue', width=2)
        )

```

1.4.2 Rendering Parameters

- **Display window:** Default 10 seconds; configurable in toolbar
- **Amplitude scale:** Default 0.5 mV/div (clinical standard); configurable
- **Refresh rate:** 30–60 Hz (user-transparent)
- **Grid:** Major gridlines every 200 ms (horizontal) and 0.5 mV (vertical)
- **Color:** Blue curves, black background

1.4.3 Performance Considerations

To maintain real-time performance:

- Data kept in circular buffers (avoid continuous reallocation)
- Rendering occurs on separate Qt thread (GUI remains responsive)
- Downsampling applied if necessary (though typically unnecessary)

1.5 Simulation Worker Thread

File: `cardiac_sim/core/simulation_worker.py`

1.5.1 Purpose

The simulation engine runs on a separate thread to prevent GUI freezing. The `SimulationWorker` class encapsulates this threading:

```

blackclass SimulationWorker(QThread):

    sample_generated = pyqtSignal(ECGSample) # Emitted when ECG sample ready

```

```
error = pyqtSignal(str)

def __init__(self):
    super().__init__()
    self._engine: SimulationEngine = ConductionGraphEngine()
    self._is_running: bool = False
    self._params: SimulationParameters = SimulationParameters()

def run(self) -> None:
    """Main simulation loop (runs on worker thread)."""
    dt = 0.002 # 2 ms per frame (500 Hz simulation rate)

    while self._is_running:
        sample = self._engine.step(dt)
        self.sample_generated.emit(sample)
        # Throttle to ~500 Hz
        time.sleep(dt / 2) # Rough throttling

def set_engine(self, new_engine: SimulationEngine, params: SimulationParameters) -> None:
    """Swap engines (must be called when NOT running)."""
    if self._is_running:
        raise RuntimeError("Cannot swap engines while running")

    self._engine = new_engine
    self._params = params
    self._engine.initialize(params)
```

1.6 Data Flow Diagram

[Diagram - text description in document]

Figure 1.2: Data flow from UI components through engines to ECG display.

Chapter 2

Detailed Implementation: Cable Engine and Cable Model

2.1 ConductionCableEngine: Complete Details

File: cardiac_sim/simulation/cable_engine.py (450 lines)

2.1.1 Overview

The `ConductionCableEngine` wraps a `ConductionCable1D` numerical model and manages beat state, ECG computation, and multi-beat rhythm.

2.1.2 Key Attributes

```
blackclass ConductionCableEngine(SimulationEngine):
    self._cable: ConductionCable1D = None
    self._params: SimulationParameters = None
    self._nodes: dict[str, ECGNode] = {}
    self._forward_model: LeadFieldModel = None

    # Beat management
    self._time: float = 0.0
    self._next_sa_fire: float = 0.0
    self._active_beats: list[BeatRecord] = [] # Up to 3 concurrent beats
    self._beat_in_progress: bool = False
    self._current_beat_nodes: dict[str, float] = {}

    # Performance tracking
    self._step_count: int = 0
```

2.1.3 Initialization: initialize

```
blackdef initialize(self, params: SimulationParameters) -> None:
    self._params = params
    self._cable = ConductionCable1D(params)
    self._nodes = _build_cable_nodes(params) # Adjusted Gaussians
    self._forward_model = LeadFieldModel()
    self._time = 0.0
    self._next_sa_fire = 0.0
    self._active_beats.clear()
```

2.1.4 Main Simulation Loop: step

This is the **critical function** that drives each simulation step:

```
blackdef step(self, dt_s: float) -> ECGSample:

    KEY PRINCIPLE: Always advance cable (never skip based on beat state).
    This ensures internal cable clock is synchronized with wall clock.
    """
    self._time += dt_s

    # --- SA pacemaker -----
    if self._time >= self._next_sa_fire:
        self._fire_sa()

    # --- Always integrate cable (CRITICAL) -----
    newly_activated = self._cable.advance(dt_s)

    # --- Collect newly activated nodes -----
    if self._beat_in_progress:
        for node_name, t_act in newly_activated.items():
            if node_name not in self._current_beat_nodes:
                self._current_beat_nodes[node_name] = t_act

        # KEY FIX: update active beat immediately
        # so ECG renders QRS at activation time
        if self._active_beats:
            self._active_beats[-1].activation_times[node_name] = t_act
```

```

# --- Check beat completion -----
self._check_beat_completion()

# --- Compute ECG -----
ecg = self._compute_ecg(self._time)

return ECGSample(timestamp=self._time, leads=ecg)

```

2.1.5 SA Pacemaker: `_fire_sa`

```

blackdef _fire_sa(self) -> None:
    cycle_len = self._params.sa_node.cycle_length_ms / 1000.0
    self._next_sa_fire += cycle_len

    # Start new beat
    beat_record = BeatRecord(
        beat_number=len(self._active_beats),
        sa_fire_time=self._time,
        activation_times={}, # Will populate as nodes activate
        qrs_start=None,
        qrs_end=None
    )
    self._active_beats.append(beat_record)
    self._beat_in_progress = True
    self._current_beat_nodes.clear()

    # Trigger cable to begin new beat
    self._cable.new_beat(self._time, self._params)

```

2.1.6 Beat Completion: `_check_beat_completion`

A beat is considered complete when:

1. All ventricular nodes have been activated, OR
2. Time exceeds $0.9 \times$ cycle length (adaptive timeout)

```

blackdef _check_beat_completion(self) -> None:
    """Commit beat when all nodes activated or timeout reached."""
    if not self._beat_in_progress or not self._active_beats:
        return

```



```

beat = self._active_beats[-1]
cycle_len = self._params.sa_node.cycle_length_ms / 1000.0

# All ventricular nodes activated?
ven_nodes = set(VENTRICULAR_NODES) # pre-defined
activated = set(beat.activation_times.keys())
ven_complete = ven_nodes.issubset(activated)

# Timeout check
time_limit = beat.sa_fire_time + 0.9 * cycle_len
timeout_reached = self._time >= time_limit

if ven_complete or timeout_reached:
    beat.is_committed = True
    self._beat_in_progress = False

# Store QRS timing for diagnostics
if beat.activation_times:
    times_v = [
        t for n, t in beat.activation_times.items()
        if n in VENTRICULAR_NODES
    ]
    if times_v:
        beat.qrs_start = min(times_v)
        beat.qrs_end = max(times_v)

```

2.1.7 ECG Computation: `_compute_ecg`

```

blackdef _compute_ecg(self, t_s: float) -> np.ndarray:

```

Returns:

np.ndarray of shape (12,) with voltages in mV

"""

```

dipole_3d = np.zeros(3) # X, Y, Z components

```

```

# Retention window: only beats within last 0.85 s contribute

```

```

retention_window = 0.85

```

```

t_cutoff = t_s - retention_window

```

```

for beat in self._active_beats:
    if beat.sa_fire_time < t_cutoff:
        # Remove old beats to prevent memory growth
        self._active_beats.remove(beat)
        continue

    # Sum contributions from all nodes
    for node_name, t_act in beat.activation_times.items():
        if node_name not in self._nodes:
            continue

        node = self._nodes[node_name]

        # Depolarization Gaussian
        tau_depol = node.depol_duration # typically 12.7 ms
        g_depol = np.exp(-(t_s - t_act)**2 / (2 * tau_depol**2))
        if t_s < t_act:
            g_depol = 0

        # Repolarization Gaussian
        t_repol = t_act + node.repol_delay # peak at ~280 ms post-act
        tau_repol = node.repol_duration
        g_repol = np.exp(-(t_s - t_repol)**2 / (2 * tau_repol**2))

        # Dipole contribution
        dipole_node = node.dipole_vector * (
            g_depol - node.repol_amplitude * g_repol
        )
        dipole_3d += dipole_node

    # Project onto 12 lead vectors
    ecg_12 = self._forward_model.project(dipole_3d)
    return ecg_12

```

2.2 ConductionCable1D: Numerical Details

File: cardiac_sim/core/tissue/cable_1d.py (500 lines)

2.2.1 Cell Configuration

```

black# Fixed anatomical structure
_NV = 10 # Ventricular cells: His(0) + LBB(1-3) + LV(4-9)

# Cell segmentation (maps to nodes)
_ATR_SEGS = {
    'SA': (0, 1),          # SA node (index 0)
    'RA_EARLY': (1, 2),    # RA early
    'RA_MID': (2, 3),
    'RA_LATE': (3, 4),
    'LA_EARLY': (4, 5),    # LA early
    'LA_MID': (5, 6),
    'LA_LATE': (6, 8),     # LA late (indices 6-7)
}

_VEN_SEGS = {
    'HIS': (0, 1),
    'LBB_PROX': (1, 2),
    'LBB_MID': (2, 3),
    'LBB_DIST': (3, 4),
    'LV_SEPT': (4, 5),     # LV septum (early)
    'LV_ANT': (5, 6),
    'LV_LAT': (6, 7),      # LV lateral (late, ~645 ms)
    'LV_INF': (7, 8),
    'LV_POST': (8, 9),     # LV posterior
    'RV': (9, 10),         # Actually computed separately; not in sequential cable
}

```

2.2.2 Diffusion Coefficients (Phase 3-D Calibrated)

```

black# Atrial cable
0.0,      # SA node (point source, no diffusion)
2.88,     # RA
2.88,
2.88,
2.88,     # LA
2.88,
2.88,
2.88

```

```
])
```

```
# Ventricular cable
```

```
_D_VEN = np.array([
    0.0,      # His bundle (point source)
    72.0,     # LBB proximal (fast conduction, D_LV=3 earlier made QRS too wide)
    72.0,     # LBB mid
    72.0,     # LBB distal
    3.0,      # LV begins
    3.0,      # LV anterior
    3.0,      # LV lateral
    3.0,      # LV inferior
    3.0,      # LV posterior
    3.0       # RV auxiliary
])
```

```
# CFL check at initialization:
```

```
D_max_atr = 2.88
```

```
D_max_ven = 72.0
```

```
dt_int = 0.0001    # 0.1 ms micro-step
```

```
tau = 0.015        # 15 ms physical time scale
```

```
CFL_atr = D_max_atr * dt_int / (tau * 1**2) = 2.88 * 0.0001 / 0.015 = 0.0192
```

```
CFL_ven = D_max_ven * dt_int / (tau * 1**2) = 72.0 * 0.0001 / 0.015 = 0.480    (margin)
```

2.2.3 FHN State Variables

```
blackclass ConductionCable1D:
```

```
    # Atrial state: (cells, [voltage, recovery])
```

```
    self._sa = np.zeros((_NA, 2)) # Shape: (8, 2)
```

```
    self._sv = np.zeros((_NV, 2)) # Shape: (10, 2)
```

```
    # Activation tracking
```

```
    self._t_act_a = np.full(_NA, math.inf) # activation time per cell
```

```
    self._t_act_v = np.full(_NV, math.inf)
```

```
    # Internal clock
```

```
    self._t = 0.0
```

```

# Stimulus timing
self._stim_end_a = 0.0
self._av_pending_at = math.inf # when AV should fire

# Pathology state
self._av_conductance = params.av_node.conductance # 0-1
self._sep_conductance = params.ventricular.septal_conductance
self._rbb_conductance = params.ventricular.rbb_conductance
self._lbb_conductance = params.ventricular.lbb_conductance

```

2.2.4 New Beat: `new_beat`

```

blackdef new_beat(self, sa_fire_time: float, params: SimulationParameters) -> None:

```

```

    Initial conditions: all cells at rest, with stimulus applied
    to SA node.

```

```

    """

```

```

    self._t = sa_fire_time

```

```

# Initialization

```

```

V_REST, W_REST = FitzHughNagumoCell.get_resting_state()

```

```

self._sa[:, 0] = V_REST # voltage

```

```

self._sa[:, 1] = W_REST # recovery

```

```

self._sv[:, 0] = V_REST

```

```

self._sv[:, 1] = W_REST

```

```

# Mark activation times

```

```

self._t_act_a[:] = math.inf

```

```

self._t_act_v[:] = math.inf

```

```

self._t_act_a[0] = sa_fire_time # SA node fires at designated time

```

```

# Stimulus window

```

```

self._stim_end_a = sa_fire_time + _STIM_DUR

```

```

self._av_pending_at = math.inf

```

```

# Refractory enforcement

```

```

self._last_activation_v = np.full(_NV, -math.inf)

```

2.2.5 Integration Step: advance

```
blackdef advance(self, dt_s: float) -> dict[str, float]:
```

Returns:

dict mapping node names to activation times for nodes that crossed threshold during this step.

```
"""
n_micro = max(1, round(dt_s / _DT_INT))
dt_micro = dt_s / n_micro
newly_activated = {}

for _ in range(n_micro):
    self._t += dt_micro

    # Atrial and ventricular integration
    self._step_atrial(dt_micro)
    self._step_ventricular(dt_micro)

    # Detect threshold crossings (activation)
    for node_name, (i0, i1) in _ATR_SEGS.items():
        v_max = np.max(self._sa[i0:i1, 0])

        # If crossed threshold and not yet activated
        if v_max > V_THRESHOLD and self._t_act_a[i0] == math.inf:
            self._t_act_a[i0] = self._t # Record activation time
            newly_activated[node_name] = self._t

    # Similar for ventricular nodes
    ...

return newly_activated
```

2.2.6 Numerical Integration Functions

```
blackdef _step_atrial(self, dt: float) -> None:
    V = self._sa[:, 0].copy()
    W = self._sa[:, 1].copy()

    # Stimulus current
```

```

I = np.zeros(_NA)
if self._t < self._stim_end_a:
    I[0] = _STIM_AMP # Stimulus only at SA node

# FHN step
V_new, W_new = _fhn_cable_step(
    V, W, _D_ATR, I, dt,
    _TAU, _A, _B, _EPS
)

self._sa[:, 0] = V_new
self._sa[:, 1] = W_new

@numba.njit(cache=True) # JIT compile if Numba available
def _fhn_cable_step(V, W, D, I_ext, dt, tau, a, b, eps):
    """FHN dynamics on cable.

    Implements discretized equations (1)-(2) with finite-difference Laplacian.
    """
    N = len(V)

    # Laplacian (no-flux boundary conditions)
    lap = np.zeros(N)
    lap[1:-1] = V[:-2] - 2.0 * V[1:-1] + V[2:]
    lap[0]     = V[1] - V[0]
    lap[-1]    = V[-2] - V[-1]

    # FHN dynamics
    inv_tau = 1.0 / tau
    dv_dt = (D * lap + V - V**3 / 3.0 - W + I_ext) * inv_tau
    dw_dt = eps * (V + a - b * W) * inv_tau

    return V + dt * dv_dt, W + dt * dw_dt

```

Note: Numba JIT attempt occurs at import time; if incompatible NumPy version, falls back to pure NumPy (3–5× slower but functional).